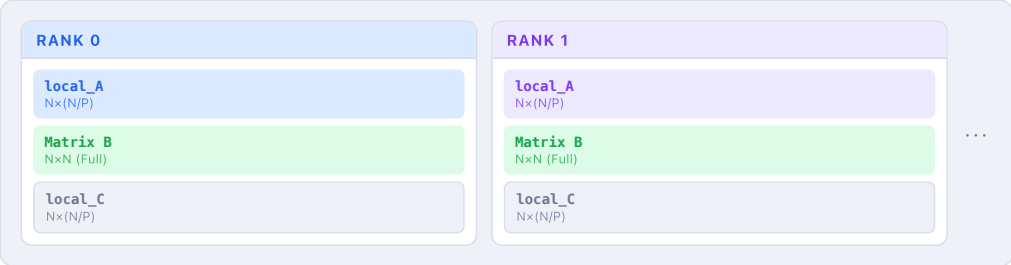
Slide 2: Memory Management & Initialization

Speaker: Ehtesham Faraz

Distributed Memory Map

local_A Full B

Allocation strategy per rank — each rank allocates only what it needs.



</> mpi_matmul.c — initialization

```c
/* Each rank generates its own A rows */
for (int i = 0; i < my_rows; i++) {
    int global_i = my_offset + i; /* global row index */
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_i, j) + seed + 0;
        local_A[i * n + j] = my_rand(&state, 0, 1);
    }
}

/* Each rank generates its own B slice */
for (int i = 0; i < my_rows; i++) {
    int global_i = my_offset + i; /* +1 offset separates A from B */
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_i, j) + seed + 1;
        local_B_slice[i * n + j] = my_rand(&state, 0, 1);
    }
}
```

Data Structures

local_A — DISTRIBUTED

Only the slice needed is allocated: `N × (N/P)`. Memory scales linearly with number of nodes.

local_B_slice — DISTRIBUTED

Each rank generates its own rows of B locally. Freed after `MPI_Allgatherv` assembles the full B.

B (full) — REPLICATED

Full N×N matrix required for multiplication. Every rank holds a complete copy after the Allgatherv phase.

Initialization Strategy

Local Generation

Data is generated locally on each node. Avoids the bottleneck of generating on Rank 0 and scattering large data over the network.

Consistent Seeding

Uses global indices `my_offset + i` to seed RNG. Matrix A uses offset `+0`, Matrix B uses `+1` — ensures independence and reproducibility.

Constraint: Verification

Only `Rank 0` allocates the full result matrix C to prevent memory overflow on worker nodes. All others discard their slice after Gatherv.

MEMORY / NODE	INIT TIME
$O(N^2/P + N^2)$	$O(N^2/P)$
slice + full B replica	per rank, fully parallel